

Musterlösung — Übungsklausur 5

Deep Learning 1 · TU Berlin

1 Multiple Choice

1. Parameter `torch.nn.Conv2d(1, 8, 3)`?

✓ (b) 80 ($= 8 \times 1 \times 3 \times 3 + 8 = 72 + 8$)

- (a) 72 — vergisst Bias
- (c) 24
- (d) 64

2. Holdout Validation?

✓ (b) Validierungsset zum Vergleich verschiedener Modelle/Hyperparameter zurückhalten

- (a) Alle Daten trainieren
- (c) Ensemble
- (d) Regularisierungsterm

3. ReLU?

✓ (b) $\text{ReLU}(x) = \max(0, x)$ — erzeugt Sparsität in Aktivierungen

- (a) Überall differenzierbar
- (c) Symmetrisch
- (d) Sättigt für große Werte

4. Konditionszahl κ ?

✓ (a) Verhältnis größter zu kleinster Eigenwert der Hessematrix

- (b) Anzahl Schichten
- (c) Train-Test-Gap
- (d) Lernrate

5. Transfer Learning?

✓ (b) Wiederverwendung von Feature-Repräsentationen aus vortrainiertem Netz

- (a) Zwei Netze gleichzeitig
- (c) Gewichte zwischen identischen Architekturen
- (d) Dataaugmentation

6. Denoising Autoencoder Vorteil?

✓ (b) Lernt Invarianzen gegenüber Rauschen und lokalen Korruptionen

- (a) Schneller
- (c) Keine Encoder-Decoder Struktur
- (d) Maximiert Rekonstruktionsfehler

7. Adversarielle Beispiele?

✓ (b) Gezielt konstruierte Inputs, kaum wahrnehmbare Perturbation → Modell macht Fehler

- (a) Falsch gelabelte Trainingsbeispiele
- (c) Ausreißer
- (d) Out-of-domain Testbeispiele

8. Texture Bias bei CNNs?

✓ (a) CNNs bevorzugen texturbasierte Features gegenüber Formfeatures

- (b) Ignorieren Farbe
- (c) Invariant gegenüber Textur
- (d) Overfit auf Formen

2 Theorie — Convolution-Operationen

(a) 2D Kreuzkorrelation

$$(x \blacksquare w)(i, j) = \sum \sum x(i+p, j+q) \cdot w(p, q)$$

Unterschied zur echten Convolution: Convolution spiegelt den Kernel ($w(-p, -q)$), Kreuzkorrelation nicht. Da Kernel gelernt werden, ist die Spiegelung bei CNNs irrelevant.

(b) Parameteranzahl Conv2d

$$\text{Gewichte: } d_{\text{out}} \times d_{\text{in}} \times k \times k = 16 \times 3 \times 5 \times 5 = 1200$$

$$\text{Bias: } d_{\text{out}} = 16$$

$$\text{Gesamt: } 1216 \text{ Parameter}$$

(c) Stride und Padding

Stride s : Schrittweite des Kernels. Stride=1: überlappende Patches, große Ausgabe. Stride=2: halbe Ausgabegröße.

$$\text{Ausgabegröße: } \text{floor}((H - k + 2 \cdot p) / s) + 1$$

Stride=1: $(H-k+2p)+1$. Stride=2: ca. Hälfte davon. Padding=0, $k=3$, $H=32$: Stride=1 $\rightarrow 30$, Stride=2 $\rightarrow 15$.

3 Theorie — Klassifikationslosses

(a) Perceptron Loss

$$\blacksquare(y, t) = \max(0, -y \cdot t)$$

Vorteil vs. 0/1: Gradient ist für Fehlklassifikationen nonzero ($\propto y$), zeigt Anpassungsrichtung. Nachteil: Training stoppt sobald alle Punkte korrekt — kein Margin, schlechte Generalisierung.

(b) Log-Loss als Regularisierer

$$\blacksquare(y, t) = \log(1 + \exp(-y \cdot t))$$

Log-Loss ist auch für korrekt klassifizierte Punkte >0 , wenn $y \cdot t$ klein. Wächst für Punkte nahe der Grenze. Das Netz muss korrekte Punkte weit von der Entscheidungsgrenze wegbewegen $\rightarrow$ impliziter Margin $\rightarrow$ bessere Generalisierung.

(c) Nicht-uniforme Fehlerkosten

Ersetze die 1 im 0/1-Loss durch tatsächliche Kosten $\text{cost}(i|k)$. Erwartete Kosten für Klasse i : $\sum_k \text{cost}(i|k) \cdot p(y=k)$. Entscheidung: $\arg \min_i \sum_k \text{cost}(i|k) \cdot p(y=k)$.

Beispiel: Medizin — falsches Negativ (krank $\rightarrow$ gesund) kostet 100, falsches Positiv (gesund $\rightarrow$ krank) kostet 5.

4 Theorie — Overfitting und Generalisierung

(a) Zwei Faktoren, die Train-Test-Gap reduzieren

$\rightarrow$ 1. Niedrige VC-Dimension (reduzierte Modellkomplexität)

$\rightarrow$ 2. Mehr Trainingsdaten N

(b) PCA zur Komplexitätsreduktion

PCA extrahiert die k Hauptkomponenten (größte Varianz) aus hochdimensionalen Daten. Das Netz wird dann auf der niedrigdimensionalen Repräsentation trainiert. Dadurch sinkt die effektive VC-Dimension des Gesamtmodells, da der Eingaberaum kleiner ist.

(c) Adversarielle Komponenten

Definition: Gezielt konstruierte Inputs, die das Modell zum schlechtestmöglichen Verhalten bringen, obwohl das Netz im Durchschnitt gut performt.

Ursache 1: Hohe Dimensionalität — ermöglicht fein abgestimmte Muster, auf die das Netz stark reagiert.

Ursache 2: Hohe Tiefe und übermäßiger Einsatz von Nichtlinearitäten — Funktion ist lokal steiler als nötig.

5 Programmierung — Autoencoder

(a) Autoencoder Klasse

```
class Autoencoder(nn.Module):
    def __init__(self):
        super().__init__()
        self.encoder = nn.Sequential(
            nn.Linear(784, 64), nn.ReLU(),
            nn.Linear(64, 32), nn.ReLU())
        self.decoder = nn.Sequential(
            nn.Linear(32, 64), nn.ReLU(),
            nn.Linear(64, 784), nn.Sigmoid())
    def forward(self, x):
        return self.decoder(self.encoder(x))
```

(b) Trainings-Loop

```
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
criterion = nn.MSELoss()
for epoch in range(epochs):
    for x, _ in dataloader: # Labels ignoriert
        x = x.view(-1, 784) # Flatten
        optimizer.zero_grad()
        x_hat = model(x)
        loss = criterion(x_hat, x) # Rekonstruktionsfehler
        loss.backward()
        optimizer.step()
```